FloodWatch

District-level flood onset prediction for India — data sources, preprocessing, model training and evaluation

This document records what the system actually does, so that every figure quoted about it can be traced back to a file. Settings come from `config.yaml`; procedures from the modules named in each section; results from `outputs/reports/summary_h1.json`. The configuration described throughout is the one that produced the committed results: 22 districts across two river basins (Kerala and the Brahmaputra), 2000–2023, predicting one day ahead. The district table in `src/data/districts.py` carries 75 districts across 12 basins and the pipeline runs on any subset; the evaluated slice is smaller because the hosted deployment has about 1 GB of memory and the full frame needs roughly 504 MB before training begins.

1 Where the data comes from

Four layers, from three providers. None requires payment or registration, and all permit academic use with attribution.

Layer	Product	Access route	Resolution / form	Licence
Rainfall (history)	ERA5 reanalysis, daily `precipitation_sum`	Open-Meteo Archive API `archive-api.open-meteo.com/v1/archive`	~25 km grid, daily, 2000–2023	Free, non-commercial, attribution
Rainfall (live)	Analysis + numerical weather prediction forecast	Open-Meteo Forecast API `api.open-meteo.com/v1/forecast`	92 days back, 16 days forward	Free, non-commercial, attribution
River discharge	GloFAS daily `river_discharge` (m^3/s)	Open-Meteo Flood API `flood-api.open-meteo.com/v1/flood`	Grid cell containing each district point	Free, non-commercial, attribution
Terrain	Elevation, from which slope and ruggedness are derived here	Open-Meteo Elevation API `api.open-meteo.com/v1/elevation`	5-point cross per district, ±5.5 km	Free, attribution
Flood labels	Dartmouth Flood Observatory event polygons (v0.9.0)	Zenodo GeoPackage `doi 10.5281/zenodo.19288170`	Event footprints with begin and end dates	CC0 (public domain)

Fetching is implemented in `src/data/download.py` *(history) and* `src/live/fetch.py` *(live). Requests carry a descriptive User-Agent, pause 1.2 s between calls, and retry four times with exponential backoff.*

1.1 What these products are, stated precisely

ERA5 and GloFAS are **reanalysis**: physics-based models that assimilate large volumes of real observation — surface stations, satellites, radiosondes — to produce a gridded, gap-free record. They are real data in the sense that matters. They are not synthetic, and they are standard inputs in published hydrology research. They are also **not IMD station gauges**. At roughly 25 km, ERA5 smooths local extremes and will systematically under-represent the peak intensity of a cloudburst. GloFAS discharge is modelled routing, not a stage reading at a gauge — which is why the column is named `river_discharge_m3s` rather than anything resembling water level.

This ceiling is real and it is worth stating before any metric is quoted: a label derived from a coarse event polygon, sampled at a single point per district, against rainfall that has had its extremes smoothed, cannot support a very high precision. A model claiming one on this data would be leaking, not learning.

1.2 How districts are located

Each district is one representative latitude/longitude, listed with its state and river basin in `src/data/districts.py`. Two coastal districts (Chennai, Cuddalore) were moved inland from their

administrative headquarters because the GloFAS grid cell at the coastal point returned no discharge at all. That is a sampling decision with a consequence — a single point stands in for a whole district — and it belongs in the limitations, not hidden in a data file.

2 Preprocessing

2.1 Schema quarantine

`src/data/loaders.py` is the only place raw files are read. Each of the four tables has a declared canonical schema, and a file that does not match it is rejected by name rather than flowing downstream to produce a confusing error three modules later. This is also what lets the same pipeline run against a synthetic fixture, the raw downloads, or the committed joined dataset without any other code knowing which.

2.2 Missing values — a stated policy, not a blanket fill

Nothing is filled with zero or with a global mean. The three layers are treated differently because they fail differently.

Layer	Policy	Why
Rainfall	Linear interpolation within a district, gaps of ≤ 2 days only. Longer gaps stay NaN and the row is dropped.	A missing rainfall day is not a zero-rainfall day. Filling it with zero fabricates dry spells, which is exactly the signal the model is trying to read.
River discharge	Same, limit 3 days.	Gauge and model dropouts are common, and discharge is strongly autocorrelated, so a short bridge is defensible where a long one is not.
Terrain	Must be complete. A missing value raises an error.	Terrain is static. If it is missing, something is wrong with the download, and silently continuing would hide that.

A `was_imputed` flag is carried per row so the share of interpolated values can be reported honestly. The flags are excluded from the feature set. Implemented in `src/data/build_dataset.py`.

2.3 Labelling — a spatial join, not text matching

The DFO release identifies an event by country, so matching on district names is impossible: every Indian event would reduce to “India”. Labels are therefore produced geometrically. Event polygons are read from the GeoPackage and reprojected to EPSG:4326; each district point is tested for containment inside each event footprint with `geopandas.sjoin(predicate="within")`; a hit is expanded into one labelled row per day between the event's begin and end dates, clipped to the study window. Districts that match no event contribute only negatives, which is correct and is noted in the console output.

Begin and end column names have changed across DFO releases (“Began”/“Ended”, “Start Date”/“End Date”, now “BeginDate”/“EndDate”), so the matcher normalises case, spaces and underscores rather than assuming one spelling.

2.4 The prediction target

The target is **onset within a window**: does a flood **begin** in this district during the seven days starting at t + 1? Onset is defined per district as a flood day whose previous day was not a flood day, so a continuing flood does not re-trigger.

Why not the obvious target

The natural choice — is a flood in progress at t + horizon — broke the evaluation, and finding out why was the most consequential decision in the project. DFO changed how long it keeps an event open: events in 2000–2014 average 20.0 days, events in 2017–2023 average 8.6 days, with the same number of events per year and comparable polygons. Only the duration bookkeeping changed. Training on “ongoing” therefore learns 20-day floods and is tested on 8.6-day floods, which is why the positive rate collapsed from 7.16% in train to 1.45% in test. That is a labelling artefact wearing the costume of a hard problem. A flood's start date is a fact about the world; its recorded duration is bookkeeping. Switching to onset removed the drift — and it is also the operationally useful question, since warning is wanted before the water arrives, not confirmation that it is still there on day twelve.

2.5 Feature engineering

59 features are built from two measured series, the terrain attributes and the calendar. The count breaks down as follows; **Appendix A lists all 59 by name**.

Group	n	Features
Raw	2	rainfall_mm, river_discharge_m3s
Lags	10	Each series at 1, 2, 3, 5 and 7 days
Rolling	20	Windows of 3, 7, 14 and 30 days: mean and max for both series, plus sum for rainfall
Antecedent wetness	1	15-day cumulative rainfall — how saturated the ground already is. This is the interaction that decides whether a given rainfall floods or drains
Rate of change	2	1-day and 3-day discharge difference. A rising river is more dangerous than a high static one
Climatology departure	3	Day-of-year rainfall climatology, plus absolute and ratio departure from it
Seasonality	3	doy_sin, doy_cos (cyclical, so December and January are adjacent), is_monsoon
Basin context	7	Same-day basin mean and max for both series, each district's ratio to its basin, and basin mean antecedent rainfall — a cheap proxy for upstream state
Percentile rank	3	Where today sits in this district's own training-period distribution, for rainfall, discharge and antecedent rainfall
Basin dummies	2	One-hot river basin (Kerala, Brahmaputra in this configuration)
Terrain	3	mean_elevation_m, mean_slope_deg, terrain_ruggedness_m
Terrain interaction	3	rain / (slope+1), antecedent × ruggedness, 1 / (elevation+10)
Total	**59**	

Two of these deserve a sentence. **Percentile rank** exists because discharge spans two orders of magnitude between basins: 800 m^3/s is a drought on the Brahmaputra and a disaster on a Konkan stream, and a model fed raw magnitudes has to learn that mapping through terrain, which it does badly. **Basin dummies** are deliberately coarse: basin, not district. District identity as a feature would be memorisation dressed up as signal, and it would collapse the first time the system was pointed at a district it had never seen.

2.6 Leakage controls

These are enforced by assertions in `tests/test_no_leakage.py` and `tests/test_live.py` rather than left as good intentions. Eighteen tests pass.

Control	What it prevents
Every rolling window uses `closed="left"`	Today's own value entering the window that is supposed to describe its past
All lag and rolling operations grouped by district	One district's history bleeding into another's features
Climatology fitted on the training split only	Test years informing the baseline they are then measured against
Percentile reference distributions fitted on the training split only	A test-year value helping to define its own percentile
NaN fill uses training-period medians	Test-set statistics leaking in through imputation
Splits are strictly chronological, asserted disjoint and ordered	The future being used to predict the past — what a random shuffle would do to a time series
Operating threshold chosen on validation, then frozen	Tuning the decision rule on the data used to report performance
Train-time references saved to `feature_reference.pkl` and reloaded at serving time	Train/serve skew: live scoring silently re-deriving its own scaling and feeding the model inputs it never saw in training

3 Training

3.1 The split

Strictly time-ordered, never shuffled. The boundaries were chosen so that the anchor event (Kerala, August 2018) falls in test while train still spans fifteen years — enough for sparse DFO events to appear in every window.

Split	Period	Rows	Positive rate
Train	2000-01-06 → 2014-12-31	120,428	1.67%
Validation	2015-01-01 → 2016-12-31	16,082	1.44%
Test	2017-01-01 → 2023-12-30	56,210	1.62%
Total		**192,720**	**1.63%**

132 rows were dropped for insufficient lag history at the start of each district's series. Validation is used for one purpose only: choosing the alert threshold.

A guard in `src/models/train_risk.py` refuses to train if any split contains zero positives, printing the class counts and the three ways to fix it. This is the normal failure mode with real flood archives — large events are sparse and can miss a window entirely — and the guard exists because the alternative is a cryptic scikit-learn error and the temptation to shuffle the split, which would leak the future.

3.2 Models

All models see the same features and the same split, so the comparison is fair. Logistic regression is included as the baseline that has to be beaten — if the ensembles do not beat it, that is a finding to report, not a failure to hide.

Model	Configuration	Imbalance handling
Logistic regression	StandardScaler → LogisticRegression, `max_iter=2000`	`class_weight="balanced"`
Random forest	`n_estimators=400, max_depth=12, min_samples_leaf=5`	`class_weight="balanced"`

Model	Configuration	Imbalance handling
XGBoost (optional)	`n_estimators=500, max_depth=6, lr=0.05, subsample=0.8, colsample_bytree=0.8`	`scale_pos_weight`, computed from the training split only

`random_state=42` *throughout. XGBoost is skipped with a printed message if the library is unavailable, so the pipeline never half-fails; the results in section 4 are from a run without it.*

Class imbalance is handled by reweighting rather than by resampling. At a 1.63% positive rate, oversampling floods or undersampling quiet days would distort the base rate the probabilities are supposed to express, and the calibration reported in section 4.4 would become meaningless. Reweighting changes what the loss function cares about while leaving the data intact.

4 Evaluation metrics

Metrics are listed in order of how much they matter for a warning system, which is not the order a classification tutorial would use. Implemented in `src/models/evaluate.py`.

4.1 Lead time and alarm duty cycle — the headline pair

For each recorded flood event, lead time is the number of hours between the first threshold crossing in the unbroken run of alarm days leading into onset, and the onset itself. An event with no alarm on the day before onset counts as a miss even if an isolated spike occurred a week earlier, because that is what an operator would actually experience.

Lead time alone is gameable: a model that leaves the alarm on all monsoon shows enormous lead time and is operationally useless. It is therefore always reported beside the **alarm duty cycle** — the share of district-days on which the alarm is raised. A good system has long lead time *and* a low duty cycle. Neither number means anything on its own.

4.2 Cost-sensitive threshold

The operating threshold is not 0.5. It is chosen on the validation split to minimise expected cost at a configured ratio of **20:1**, missing a flood against raising a false alarm (`cost_false_negative: 20`, `cost_false_positive: 1` in config.yaml). The curve is swept across 99 thresholds and the minimum is taken.

This produces **low precision by design**. Saying that twenty false alarms are worth one additional flood caught is the same as saying most alarms will come to nothing. Because that reads as a defect when quoted alone, a second operating point — the F1-optimal threshold, weighting precision and recall equally — is reported from the same model. The pair is the honest answer; either number alone misleads in a different direction.

4.3 Discrimination

PR-AUC is reported ahead of ROC-AUC. At a 1.63% positive rate, ROC curves look flattering because the true-negative pool is enormous, while precision-recall curves stay honest about how rare the positives are. The confusion matrix at the chosen threshold is given in full, and recall is quoted before precision because recall is the number that corresponds to people not being warned.

4.4 Calibration

Reported as a Brier score plus a ten-bin reliability curve. If the model says 30%, floods should begin on roughly 30% of such days. An uncalibrated score cannot be acted on by a person, and for this system calibration matters more than ranking, because an officer responds to the number rather than to the ordering.

4.5 Results

Held-out test split, 2017–2023, 130 recorded flood onsets. Threshold fixed on validation and applied unchanged.

Metric	Logistic regression	Random forest
Events warned	84 / 130 (64.6%)	66 / 130 (50.8%)
Median lead time	312 h (13.0 days)	120 h (5.0 days)
Mean lead time	236.6 h (9.9 days)	139.6 h (5.8 days)
Alarm duty cycle	14.65%	10.46%
Recall	0.545	0.393
Precision	0.060	0.061
F1	0.108	0.105
PR-AUC	0.095	0.061
ROC-AUC	0.780	0.787
Brier score	0.162	0.060
Operating threshold	0.71	0.41
Confusion (tp / fp / fn / tn)	496 / 7,740 / 414 / 47,560	358 / 5,521 / 552 / 49,779
Expected cost at 20:1	16,020	16,561

Source: outputs/reports/summary_h1.json. Lead times converted from hours for readability; the stored values are hours.

Read operationally: **logistic regression is the better warning system here**, catching roughly two-thirds of real onsets with a median of thirteen days' notice, for an alarm on about one district-day in seven. The random forest is better calibrated (Brier 0.060 against 0.162) and marginally better at ranking (ROC-AUC 0.787), but warns half the events at a third of the notice. That the simple baseline wins on the metric that matters is a result worth reporting, not an embarrassment to bury.

Why accuracy does not appear in that table

Predicting “no flood” every single day scores **98.38%** accuracy on this test set, because flood onsets begin on 1.62% of district-days. Both models score *below* that do-nothing baseline at their F1-optimal thresholds — 85.5% and 90.1% — and both are nonetheless useful, because they warn most real events many days early. Accuracy is not a weak metric at this base rate; it is an actively misleading one, and the system prints the all-negative baseline beside every accuracy figure so that the comparison cannot be avoided.

4.6 On the precision figure

A precision of roughly 0.06 deserves an explanation rather than a footnote, and it has two causes in known proportions. The first is deliberate: the 20:1 cost ratio instructs the model to accept about twenty false alarms rather than miss one onset, and precision is the price of that instruction. Moving to the F1-optimal threshold changes it very little (0.061 → 0.064 for the forest), which shows the second cause is the larger one: irreducible label noise. Coarse DFO polygons, one sample point per district, and ERA5's smoothing of extremes together put a ceiling on precision that no amount of model tuning will lift. The honest conclusion is that this system is useful for deciding *where to look*, and is not a basis for acting alone.

5 Limitations to state in the report

Limitation	Consequence
ERA5 is ~25 km and smooths extremes; GloFAS is modelled routing, not gauge stage	Peak intensities are under-represented, which caps achievable precision
One sample point stands in for an entire district	A flood affecting part of a district may be labelled wrongly in either direction
DFO records only large events	Smaller, locally significant floods are absent from the labels and count as negatives
The live feed is analysis and forecast; the model was trained on reanalysis	Train/serve skew. Live output carries more uncertainty than the backtest implies, and the gap widens with lead time. The rigorous fix needs an archive of past forecasts, which is not available
Forecast skill decays with lead time	A six-day-ahead score is built largely from forecast rainfall. Each output row carries a `forecast_share` column so this is visible rather than hidden
Evaluated on 22 districts in two basins	Results should not be assumed to transfer to basins not in the training set

Appendix A The 59 features in full

Read straight out of `features_h1.parquet` by `feature_columns()`, in the order the model receives them. Every rolling window is **left-closed**: it covers the days *before* t, never t itself, so the current day cannot enter the window that is meant to describe its own past. Everything is grouped per district, so one district's history never reaches another's row.

#	Feature	Definition
Measured series		
1	`rainfall_mm`	ERA5 daily precipitation total at the district point, in mm.
2	`river_discharge_m3s`	GloFAS daily mean river discharge for the grid cell containing the point, in m^3/s. Modelled routing, not a gauge stage reading.
Terrain (static per district)		
3	`mean_elevation_m`	Elevation at the district point, in metres. Note the column name: the stored value is the centre sample of the five-point cross, not the mean of the five.
4	`mean_slope_deg`	Slope in degrees, from the finite-difference gradient of the five elevation samples taken at ±0.05° (~5.5 km).
5	`terrain_ruggedness_m`	Standard deviation of those five elevation samples — a proxy for local relief.
Rainfall lags		
6	`rainfall_mm_lag1`	Rainfall one day earlier.
7	`rainfall_mm_lag2`	Rainfall two days earlier.
8	`rainfall_mm_lag3`	Rainfall three days earlier.
9	`rainfall_mm_lag5`	Rainfall five days earlier.
10	`rainfall_mm_lag7`	Rainfall seven days earlier.
Rainfall rolling windows		
11	`rainfall_mm_rollmean3`	Rolling mean of rainfall over the preceding 3 days.
12	`rainfall_mm_rollmax3`	Rolling maximum of rainfall over the preceding 3 days.
13	`rainfall_mm_rollsum3`	Rolling total of rainfall over the preceding 3 days.
14	`rainfall_mm_rollmean7`	Rolling mean of rainfall over the preceding 7 days.
15	`rainfall_mm_rollmax7`	Rolling maximum of rainfall over the preceding 7 days.
16	`rainfall_mm_rollsum7`	Rolling total of rainfall over the preceding 7 days.
17	`rainfall_mm_rollmean14`	Rolling mean of rainfall over the preceding 14 days.
18	`rainfall_mm_rollmax14`	Rolling maximum of rainfall over the preceding 14 days.
19	`rainfall_mm_rollsum14`	Rolling total of rainfall over the preceding 14 days.
20	`rainfall_mm_rollmean30`	Rolling mean of rainfall over the preceding 30 days.
21	`rainfall_mm_rollmax30`	Rolling maximum of rainfall over the preceding 30 days.
22	`rainfall_mm_rollsum30`	Rolling total of rainfall over the preceding 30 days.
Discharge lags		
23	`river_discharge_m3s_lag1`	Discharge one day earlier.
24	`river_discharge_m3s_lag2`	Discharge two days earlier.

#	Feature	Definition
25	`river_discharge_m3s_lag3`	Discharge three days earlier.
26	`river_discharge_m3s_lag5`	Discharge five days earlier.
27	`river_discharge_m3s_lag7`	Discharge seven days earlier.
Discharge rolling windows — mean and max only		
28	`river_discharge_m3s_rollmean3`	Rolling mean of discharge over the preceding 3 days.
29	`river_discharge_m3s_rollmax3`	Rolling maximum of discharge over the preceding 3 days.
30	`river_discharge_m3s_rollmean7`	Rolling mean of discharge over the preceding 7 days.
31	`river_discharge_m3s_rollmax7`	Rolling maximum of discharge over the preceding 7 days.
32	`river_discharge_m3s_rollmean14`	Rolling mean of discharge over the preceding 14 days.
33	`river_discharge_m3s_rollmax14`	Rolling maximum of discharge over the preceding 14 days.
34	`river_discharge_m3s_rollmean30`	Rolling mean of discharge over the preceding 30 days.
35	`river_discharge_m3s_rollmax30`	Rolling maximum of discharge over the preceding 30 days.
Accumulation and rate of change		
36	`antecedent_rain_mm`	Total rainfall over the preceding 15 days — how saturated the ground already is. This is the term that decides whether a given rainfall floods or drains.
37	`river_discharge_delta_1d`	Change in discharge since the previous day. A rising river is more dangerous than a high static one.
38	`river_discharge_delta_3d`	Change in discharge over three days.
Climatology departure		
39	`rainfall_climatology_mm`	Mean rainfall for this district on this day-of-year, fitted on the **training split only** and smoothed over a 15-day centred window.
40	`rainfall_departure_mm`	Today's rainfall minus that climatology — how unusual today is for the time of year.
41	`rainfall_departure_ratio`	Today's rainfall divided by that climatology.
Seasonality		
42	`doy_sin`	sin(2π × day-of-year / 365.25). Cyclical, so 31 December sits next to 1 January instead of 364 days away.
43	`doy_cos`	cos(2π × day-of-year / 365.25).
44	`is_monsoon`	1 for June to September, 0 otherwise.
Basin context — same day, other districts		
45	`basin_rainfall_mm_mean`	Mean rainfall across every district in the same river basin on the same day.
46	`basin_rainfall_mm_max`	Maximum rainfall anywhere in the basin that day.
47	`rainfall_mm_vs_basin`	This district's rainfall divided by its basin mean — how extreme it is relative to its neighbours.
48	`basin_river_discharge_m3s_mean`	Basin mean discharge that day.
49	`basin_river_discharge_m3s_max`	Basin maximum discharge that day.

#	Feature	Definition
50	`river_discharge_m3s_vs_basin`	This district's discharge divided by its basin mean.
51	`basin_antecedent_mean`	Basin mean of the 15-day antecedent rainfall — a cheap proxy for upstream saturation, without building a routing model.
Percentile rank within the district's own history		
52	`rainfall_mm_pctile`	Where today's rainfall sits in this district's **training-period** rainfall distribution, 0 to 1.
53	`river_discharge_m3s_pctile`	The same for discharge. This is what makes 800 m^3/s readable as a drought on the Brahmaputra and a disaster on a Konkan stream.
54	`antecedent_rain_mm_pctile`	The same for 15-day antecedent rainfall.
River basin, one-hot		
55	`basin_Brahmaputra`	1 if the district is in the Brahmaputra basin.
56	`basin_Kerala`	1 if the district is in the Kerala basin.
Terrain interactions		
57	`rain_x_inv_slope`	rainfall / (slope + 1). The same rainfall is more dangerous on flat ground that drains slowly.
58	`antecedent_x_ruggedness`	15-day antecedent rainfall × ruggedness.
59	`elevation_inv`	1 / (elevation + 10). Large for low-lying districts.

The basin one-hot has two columns because this configuration covers two basins. Running the pipeline over all 12 basins in `districts.py` *produces ten more columns and a feature count of 69; nothing else in the list changes.*

A.1 What is deliberately excluded

Columns that exist in the table and are kept out of the model on purpose. Listing them matters as much as listing the features: each one is a way the model could have cheated.

Column	Why it is not a feature
`lat`, `lon`	Coordinates would let a tree carve out individual districts and memorise their flood history. Location enters only through terrain and the basin one-hot.
`district`	District identity is memorisation dressed up as signal. A model using it would collapse the first time it was pointed at a district it had never seen.
`basin`, `state`	Kept as columns for grouping and reporting. Basin reaches the model only as the coarse one-hot above; state never does.
`date`	A raw timestamp lets a model learn the calendar of the training years. Time enters only through the cyclical day-of-year terms.
`flood_occurred`	Today's label. Feeding it alongside a target built from the same series is the most direct leak available.
`rainfall_mm_was_missing` `river_discharge_m3s_was_missing`	Imputation flags, kept so the share of interpolated rows can be reported. They describe the data-collection process, not the weather.

FloodWatch is a student project and decision support. It is not an operational warning service, and nothing it produces is an evacuation order. India's official flood forecasts come from the Central Water Commission and the India Meteorological Department.